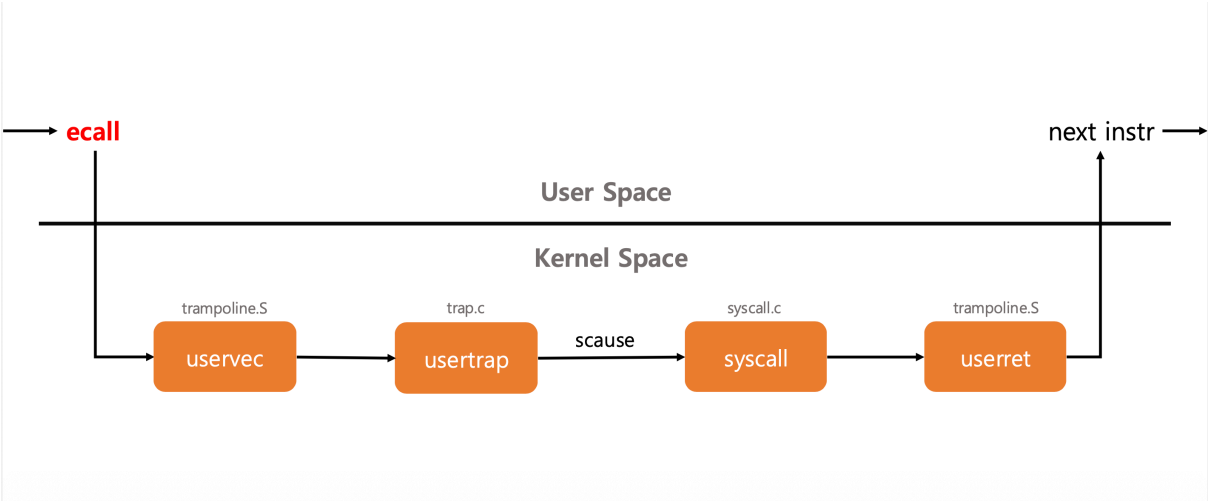
Report

Design

이번 과제는 현재 실행 중인 프로세스 목록을 표시해주는 user program과 system call을 구현하는 과제다.



Lab 3에서 구현했던 `getppid`의 system flow를 참고하여 구상할 계획을 세웠다. User mode에서 직접적으로 kernel 함수를 호출할 수 없기 때문에 `usys.S`에서 정의되어 있는 ecall 명령어가 실행되어 kernel(supervisor) mode로 전환한 뒤 kernel 함수가 실행된다. Lab 3 명세에 명시된대로 수정해야 할 코드 파일을 선택하고, `ps.c`라는 유저 프로그램을 생성하여 ps 명령어를 테스트한다.

Implementation

가장 먼저 `kernel/proc.c` 파일부터 수정하였다. `kernel/proc.c`의 `procdump` 함수 구조를 참고하여 ps 함수의 kernel mode 버전인 `ps` 함수를 다음과 같이 생성하였다.

```
692  int
693  ps(int pid)
694  {
695    static char *states[] = {
696    [UNUSED]    "unused",
697    [USED]      "used",
698    [SLEEPING]  "sleep",
699    [RUNNABLE]  "runble",
700    [RUNNING]   "run",
701    [ZOMBIE]    "zombie"
702    };
703    struct proc *p;
704    char *state;
705
706    if(pid == 0)
707    {
708      for(p = proc; p < &proc[NPROC]; p++){
709        if(p->state == UNUSED)
710          continue;
711        if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
712          state = states[p->state];
713        else
714          state = "???";
715        printf("%d,%s,%s\n", p->pid, state, p->name);
716      }
717      return 0;
718    }
719    else {
720      for(p = proc; p < &proc[NPROC]; p++){
721        if(p->state == UNUSED)
722          continue;
723        if(p->pid == pid)
724        {
725          if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
726            state = states[p->state];
727          else
728            state = "???";
729          printf("%d,%s,%s\n", p->pid, state, p->name);
730          return 0;
731        }
732      }
733      return 1;
734    }
735  }
```

procdump 함수와의 차이점은 인자로 받은 pid 값에 따라 분기한다는 점이다. pid == 0일 경우 명세에 따라 현재 실행 중인 모든 프로세스를 출력하도록 하였다. 그 외의 경우 인자로 받은 pid(`pid`)가 for문을 따라 순회 중인 proc 구조체 `p`의 pid (`p->pid`)와 일치하면 현재 실행 중인 프로세스이므로 state를 업데이트해준다. 이때 바로 0을 반환하여 함수를 종료하고 그렇지 않은 경우, 즉 인자로 받은 pid를 찾지 못한 경우 1을 반환한다. 이는 추후 `ps.c`에서의 로직과 연결된다.

```
101   int             either_copyout(int user_dst, uint64 dst, void *src, uint64 len);
102   int             either_copyin(void *dst, int user_src, uint64 src, uint64 len);
103   void            procdump(void);
104   int             ps(int);
```

`kernel/defs.h`에 위에서 구현한 `ps` 함수를 추가하여 다른 C 파일에서 `ps` 함수를 사용할 수 있도록 정의하였다.

```
111   uint64
112   sys_ps(void)
113   {
114     int pid;
115
116     argint(0, &pid);
117     return ps(pid);
118   }
```

`kernel/sysproc.c`에 wrapper 함수를 추가하였다. Lab 3의 경우 `kgetppid` 함수가 인자를 받지 않아 바로 함수를 반환값으로 사용할 수 있었는데, `ps`의 경우 int형 인자를 받기 때문에 `kernel/sysproc.c`의 `sys_kill` 함수를 참고하여 구현하였다.

```
21    #define SYS_mkdir  20
22    #define SYS_close  21
23    #define SYS_ps     22
24
```

```
102   extern uint64 sys_mkdir(void);
103   extern uint64 sys_close(void);
104   extern uint64 sys_ps(void);
105
106   // An array mapping syscall numbers from syscall.h
107   // to the function that handles the system call.
108   static uint64 (*syscalls[])(void) = {
109   [SYS_fork]    sys_fork,
110   [SYS_exit]    sys_exit,
111   [SYS_wait]    sys_wait,
112   [SYS_pipe]    sys_pipe,
113   [SYS_read]    sys_read,
114   [SYS_kill]    sys_kill,
115   [SYS_exec]    sys_exec,
116   [SYS_fstat]   sys_fstat,
117   [SYS_chdir]   sys_chdir,
118   [SYS_dup]     sys_dup,
119   [SYS_getpid]  sys_getpid,
120   [SYS_sbrk]    sys_sbrk,
121   [SYS_pause]   sys_pause,
122   [SYS_uptime]  sys_uptime,
123   [SYS_open]    sys_open,
124   [SYS_write]   sys_write,
125   [SYS_mknod]   sys_mknod,
126   [SYS_unlink]  sys_unlink,
127   [SYS_link]    sys_link,
128   [SYS_mkdir]   sys_mkdir,
129   [SYS_close]   sys_close,
130   [SYS_ps]      sys_ps,
131   };
```

`kernel/syscall.h`와 `kernel/syscall.c`에 `sys_ps` wrapper 함수를 system call로 등록하였다.

```
25   int pause(int);
26   int uptime(void);
27   int ps(int);
```

이제 user 단으로 넘어와 `user/user.h`에 `ps` 함수를 등록하여 사용자가 `ps` 함수를 사용할 수 있도록 만들었다.

```
42  entry("sbrk");
43  entry("pause");
44  entry("uptime");
45  entry("ps");
```

`user/usys.pl`에 ps 매크로를 추가하여 ecall 기능을 사용할 수 있도록 해주었다.

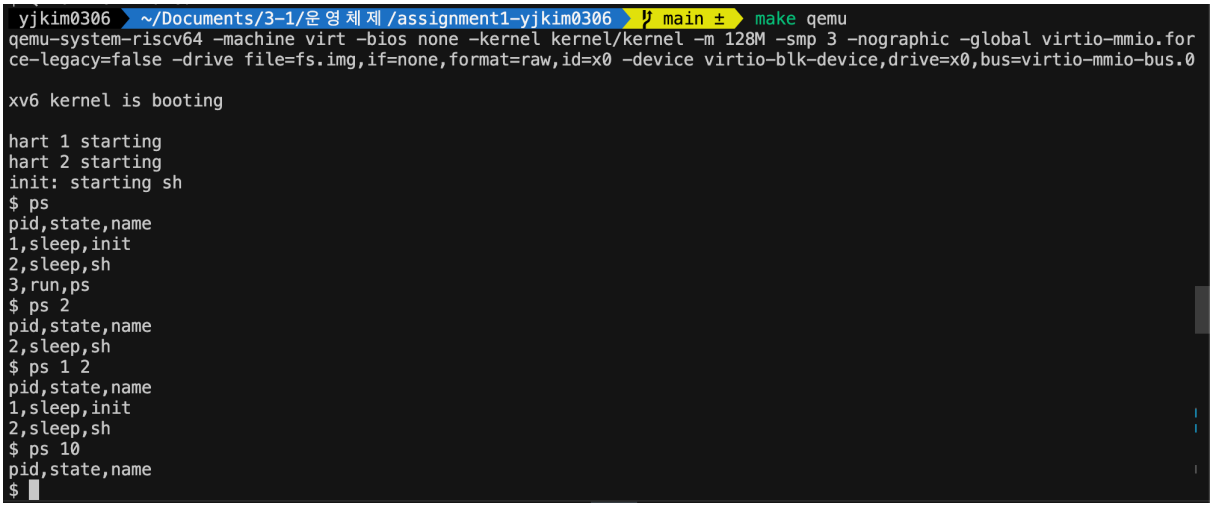
```
1   #include "kernel/types.h"
2   #include "kernel/stat.h"
3   #include "user/user.h"
4
5   int
6   main(int argc, char *argv[])
7   {
8       printf("pid,state,name\n");
9       if(argc == 1){
10          ps(0);
11      }
12      else{
13          for(int i = 1; i < argc; i++)
14              ps(atoi(argv[i]));
15      }
16    exit(0);
17  }
```

`user/ps.c`라는 유저 프로그램을 생성하여 `ps` 함수를 호출하였다. 인자의 개수 `argc`가 1인 경우 `ps` 만을 받은 경우이므로 `pid`로 0을 받은 것으로 간주하여 `ps(0)`을 실행하였다. 그 외의 경우 `ps` 뒤에 인자로 `pid` 값을 추가로 받은 경우이므로 분기하였다. 이때 받은 인자의 수만큼 iteration을 돌리고, 첫 인자인 `ps`를 제외하고 뒤에 받은 `argv` 배열의 각 char형 변수를 `atoi` 함수를 이용해 int형으로 변환해준다. 그리고 그 int형 변수를 `ps`의 매개변수로 넘겨준다. 이렇게 하면 앞서 `kernel/proc.c`의 `ps` 함수에서 한 번의 function call에는 한 번의 pid만을 검사한다는 로직이 `user/ps.c`의 `main`에서 모든 pid를 검사하는 iteration에 감싸지기 때문에 입력받은 모든 pid를 출력하게 된다.

```
146     $U/_forphan\
147     $U/_dorphan\
148     $U/_ps\
```

마지막으로 Makefile에 `ps` 명령어를 추가하여 터미널에서 `ps`를 호출할 수 있도록 하였다.

Results



명세에 나온 테스트 케이스부터 실행하였다. `make qemu` 명령어로 xv6를 실행하고 각 케이스를 입력하였다.

- ps: 현재 실행 중인 모든 프로세스 출력 → 통과
- ps 2: 2번 프로세스 출력 → 통과
- ps 1 2: 1, 2번 프로세스 출력 → 통과
- ps 10: 10번 프로세스 출력, 10번 프로세스는 존재하지 않으므로 출력하지 않음 → 통과

모두 정상적으로 출력되었다.

Troubleshooting

`kernel/proc.c`와 `user/ps.c` 사이의 로직을 구성하는 과정에서 헷갈리는 부분이 있어 고민하였다. ps 함수의 반환값은 현재 실행 중인 프로세스의 존재 여부인데, 처음에는 `proc.c`에서 모든 프로세스를 순회하고 존재 여부를 판단하여 반환값을 설정해야 한다고 생각하여 인자의 개수를 받아오는 방법을 구상하였다.

그러나 구조적으로 인자의 개수는 터미널에서 입력을 받는 부분에서 결정되고 이는 `ps.c`의 `argc`값이기 때문에 kernel 프로그램인 `proc.c`로 넘겨줄 방법이 없다. 따라서 인자의 개수 구분 로직과 인자값은 `ps.c`에서 구현하고, `proc.c`에서는 한 번의 function call에 한 번의 pid 값을 찾는 로직을 사용하여 각각의 user/kernel mode에서 접근 가능한 부분을 사용할 수 있었다.

GitHub Discussions 탭의 질문 중 흥미로운 예외 케이스가 있어 실행해보았다.

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ dorphan &
$ wait for kill and reclaim

$ ps
pid,state,name
1,sleep,init
2,sleep,sh
5,run,ps
4,sleep,dorphan
$
```

dorphan 명령어로 프로세스를 하나 만들고 ps를 실행하니 위와 같이 출력되었다. 오름차순이 아닌 5번 프로세스(ps)가 먼저 출력되고 4번 프로세스(dorphan)가 나중에 출력되었다. 이 원인을 나름대로 예측해보고자 `kernel/proc.c`의 `ps` 함수의 `pid == 0` 분기를 분석하였다. 분기의 for문에서는 proc 구조체 `p`를 순차적으로 돌면서 0번째 인덱스부터 끝까지 탐색한다. dorphan을 실행하면서 fork된 프로세스가 그 사이 종료되면서 해당 프로세스가 있던 인덱스가 비어있게 되고, 그 자리를 ps 프로세스가 차지한 것으로 예상된다.

System Call Execution Flow

1. 사용자가 터미널에 `ps` 명령어를 입력한다.
2. `user/ps.c`에 `ps`를 비롯한 인자값이 있다면 이를 받아 `ps` 함수가 실행된다.
3. `user/usys.S`에 `SYS_ps`의 번호값인 22를 `a7` register에 저장한 뒤 ecall을 실행한다.
4. ecall이 실행되면 kernel mode로 전환되고 `kernel/syscall.c`의 `syscall` 함수가 실행된다.
5. 현재 프로세스(ps)의 trapframe에서 `a7` register에 저장된 22를 읽어와 22번에 매핑되어 있는 `SYS_ps`가 실행된다.
6. `kernel/sysproc.c`에서 호출된 `sys_ps` 함수가 `argint(0, &pid)`를 통해 인자를 가져오고 `ps`에 넘겨 `ps`를 실행한다.
7. `kernel/proc.c`에 정의된 `ps` 함수가 현재 실행 중인 프로세스를 탐색하여 프로세스가 있으면 0, 없으면 1을 반환한다.
8. `kernel/sysproc.c`의 `sys_ps` 함수가 `ps`의 반환값을 받아 `kernel/syscall.c`의 `syscall` 함수로 넘겨준다.

9. `syscall` 함수가 사용자가 반환값을 받을 수 있게 이 값을 `a0` register에 저장하고 다시 user mode로 설정한다.
10. `ps.c`에서 `ps` 함수가 호출된 다음 줄부터 다시 실행되고, `exit(0)`를 만나 프로그램이 종료된다.